TEST PLAN

Project Title: DEPI Software Testing Graduation Project – E-Commerce
Track: Digital Egypt Pioneers Initiative (DEPI) – Software Testing Track
Prepared By: Abdullah Yasser, Nada Mamdouh, Tasneem Ramzy, Basma Barakat, Noor Islam, Malak Mohamed
Date: November-December 2026

1. Introduction

1.1 Purpose

This Test Plan defines the comprehensive quality assurance strategy, scope, resources, schedule, and methodologies for testing the Graduation Project under the Digital Egypt Pioneers Initiative (DEPI) Software Testing Track. It serves as the governing blueprint for the 6-person QA team—led by the Project Leader alongside five QA team members—aligning all testing activities with established Software Development Lifecycle (SDLC) and Software Testing Lifecycle (STLC) best practices.

1.2 Project Overview

The objective of this project is to validate the functional accuracy,end-to-end business workflows, and reliability of the Application Under Test (AUT) while demonstrating mastery of the full DEPI Software Testing syllabus. The testing lifecycle bridges foundational Software Testing Principles, Static Testing, Manual Testing, Black-Box Functional Testing, defect management via Jira, and an enterprise-grade Java-based Test Automation Framework leveraging Object-Oriented Programming (OOP).

2. Scope

2.1 In-Scope

The following testing levels, techniques, and technical components are within the scope of this graduation project:

- **Static Testing:** Early-stage verification of Software Requirements Specifications (SRS), user stories, architectural design documents, and peer reviews of test artifacts and test automation scripts.
- **Manual Functional Testing:** Execution of end-to-end functional test cases covering core business workflows, UI validation, Smoke Testing, Sanity Testing, System Testing, and Regression Testing.
- **Black-Box Testing Techniques:** Systematic test design using Equivalence Partitioning (EP), Boundary Value Analysis (BVA), Decision Table Testing, State Transition Testing, and Exploratory Testing.
- **Automation Testing (Java & Selenium):** Development and execution of automated regression suites built in Java using **OOP Principles** (Encapsulation, Inheritance, Polymorphism, Abstraction via the Page Object Model).
- **Defect & Project Management:** End-to-end bug tracking, requirement traceability, and sprint task management using Jira.

2.2 Out-of-Scope

Unless explicitly added via a formal change request, the following are excluded from the current project scope:

- Third-party payment gateway or external hardware infrastructure penetration testing.
- Large-scale distributed load/stress testing beyond baseline functional response checks.
- Native mobile application testing (iOS/Android), unless responsive web views are specified.

3. Test Approach

Our testing strategy integrates into an Agile SDLC model, combining Shift-Left verification with iterative manual and automated validation phases:

3.1 Static Testing Phase

Before code execution, the team conducts structured Walkthroughs, Technical Reviews, and Inspections on requirements and design artifacts to identify ambiguities, omissions, and logic defects early in the SDLC. Static analysis and peer reviews are also performed on the Java test codebase during the Project and Review milestones.

3.2 Manual & Black-Box Testing Phase

Test engineers derive structured test scenarios and test cases from user stories managed in **Jira**. Black-box design techniques are applied as follows:

- **Equivalence Partitioning (EP) & Boundary Value Analysis (BVA):** Applied to input fields, numerical ranges, and string length constraints.
- **Decision Table & State Transition Testing:** Applied to complex business rules, role-based access permissions, and multi-step workflow states.
- **Exploratory & Ad-hoc Testing:** Conducted after formal test execution to uncover edge-case defects not captured by scripted scenarios.

3.3 Test Automation Architecture (Java)

Critical smoke and regression test cases are automated using Java and Selenium WebDriver, structured around core syllabus concepts:

- **Object-Oriented Programming (OOP):** Implementation of the Page Object Model (POM) design pattern—using Encapsulation to protect WebElements with private locators and public action methods, Inheritance (BaseTest and BasePage classes), Polymorphism (method overloading for dynamic waits and actions), and Abstraction (interfaces and abstract utility classes).

4. Test Entry and Exit Criteria

4.1 Entry Criteria

- Approved SRS, user stories, and acceptance criteria are baselined and uploaded to Jira.
- The dedicated test/staging environment is deployed, stable, accessible via staging URL, and populated with required test data and accounts.
- Static testing of requirements is completed with all high-priority ambiguities resolved.
- Test cases and test data files are reviewed and approved by the Project Leader.
- Smoke test passes at 100% on the initial build.

4.2 Exit Criteria

- 100% of planned Manual and Automated high-priority test cases have been executed.
- A minimum requirement coverage of 90% is achieved across all documented functional requirements, user stories, and acceptance criteria in the Requirements Traceability Matrix (RTM).
- Zero open Critical or High/Blocker severity defects remain in Jira.
- All Medium/Low defects are documented, triaged, and have approved workarounds or deferral sign-off.
- Complete Test Summary Report, RTM, and automated test execution reports are delivered and reviewed.

4.3 Suspension and Resumption Criteria

- **Suspension:** Testing will be paused if a Blocker defect prevents access to more than 40% of core functionalities or if the staging environment experiences critical downtime.
- **Resumption:** Testing resumes once a fixed build is deployed and passes the Smoke Test suite.

5. Test Deliverables

Phase	Deliverable
Planning	Test Plan & Jira Project Board Setup
Static Testing	Requirement Review Logs & Specification Inspection Checklist
Design	Requirements Traceability Matrix (RTM), Test Scenarios, Test Cases
Automation	Java Automation Framework (OOP)
Execution	Executed Test Run Logs & Jira Defect Reports
Closure	Final Test Summary Report, Metrics Dashboard & Project Review Presentation

6. Test Environment

6.1 Hardware & Operating System Configuration

- **Workstations:** Team PCs running Windows 10 / Windows 11 (64-bit), minimum 8–16 GB RAM, multi-core processors.
- **Browsers:** Google Chrome (latest stable), Mozilla Firefox (latest stable), Microsoft Edge (latest stable),Brave (latest stable).

6.2 Software & Testing Tools Stack

- **Programming Language:** Java (JDK 17 / JDK 21)
- **Integrated Development Environment (IDE):** IntelliJ IDEA / Eclipse IDE
- **Test Automation Stack:** Selenium WebDriver.
- **Project & Defect Management:** Atlassian Jira (Agile Scrum, Bug Tracking, Test Management)
- **Version Control System:** Git & GitHub (collaborative branching, pull requests, and code reviews)

7. Roles and Responsibilities

The team comprises 6 professionals: the Project Leader and 5 QA Team Members with clearly segregated duties across manual and automation tracks:

Role / Resource	Assigned Responsibilities
Project Leader	Defines the Master Test Plan, configures and administers the Jira workspace, oversees SDLC/STLC alignment, leads code and test case reviews, architects the core Java OOP Automation Framework, tracks milestones, and compiles the Final Test Summary Report.
Manual & Static Testing Lead	Leads Static Testing reviews on SRS/user stories, creates the Requirements Traceability Matrix (RTM), designs black-box test cases (EP, BVA), and executes functional & smoke testing.
Functional & Black-Box Tester	Designs and executes functional test cases using Decision Table and State Transition techniques, conducts UI/usability validation in Jira.
Automation Engineer – Core & OOP	Develops Page Object Model (POM) classes using Java OOP principles, implements WebElement locators and dynamic synchronization, and utilizes the Java Collections Framework for multi-element verification.

8. Schedule

The graduation project follows a structured 4-week STLC timeline:

Phase / Milestone	Key Activities	Target Timeline
Week 1: Requirement Analysis & Planning	Finalize Test Plan, set up Jira board, perform Static Testing on requirements, and establish Git repository.	Week 1
Week 2: Test Design & Framework Setup	Design Manual Test Cases (Black-Box), prepare Test Data, and build base Java OOP Automation architecture (POM, File I/O, Collections).	Week 2
Week 3: Test Execution & Automation Scripting	Execute Cycle 1 Manual Tests, log defects in Jira, develop automated test scripts.	Week 3
Week 4: Regression, Closure & Final Review	Verify bug fixes, run automated Regression Suite, conduct Project & Test Automation Framework Review, compile Test Summary Report, and prepare final graduation presentation.	Week 4

9. Risks and Mitigation Strategies

Risk Description	Probability	Impact	Mitigation Strategy
Incomplete or Ambiguous Requirements	Medium	High	Conduct rigorous Static Testing walkthroughs in Week 1 and maintain a formal Q&A log with stakeholders/instructors in Jira.
Unstable Test Environment or Late Build Delivery	Medium	High	Enforce strict Entry Criteria (Smoke Test gate) and modularize the Java Automation Framework using config files for rapid environment switching.
Automation Script Flakiness (Dynamic UI Elements)	High	Medium	Implement resilient dynamic waits and clean OOP Page Object Model locators rather than brittle hardcoded XPaths.
Schedule Compression Near Submission Deadline	Medium	High	Prioritize test cases by risk (High/Critical executed first) and leverage the automated regression suite to minimize manual re-testing effort.
Merge Conflicts in Shared Automation Repository	Low	Medium	Enforce strict Git branching rules, modular package structures, and mandatory peer reviews led by the

Risk Description	Probability	Impact	Mitigation Strategy
			Project Leader before merging to main.